

Degree name
Month Year



Reliability in Event-Driven Design

How Correctness Boundaries Shape Process-Handling
Choices and Recovery Behaviour

Firstname1 Lastname1

Firstname2 Lastname2

This thesis is submitted to the Faculty of Faculty at Blekinge Institute of Technology in partial fulfillment of the requirements for the degree of Degree name. The thesis is equivalent to Weeks weeks of full-time studies.

The authors declare that they are the sole authors of this thesis and that they have not used any sources other than those listed in the bibliography and identified as references. They further declare that they have not submitted this thesis to any other institution to obtain a degree.

Contact Information:

Author(s):

Firstname1 Lastname1

E-mail: ...@student.bth.se

Firstname2 Lastname2

E-mail: ...@student.bth.se

University advisor:

Title Firstname Lastname

Department of Department

Faculty of Faculty
Blekinge Institute of Technology
SE-371 79 Karlskrona, Sweden

Internet : www.bth.se
Phone : +46 455 38 50 00
Fax : +46 455 38 50 57

Abstract

Background. Reliability in event-driven processes is difficult to judge when the condition a process must protect is left implicit. Asynchronous handling can preserve, delay, replay, duplicate, or omit work, but these behaviours have different significance depending on what still has to hold for a case to count as acceptable. Without that prior correctness condition, the value of a handling choice becomes harder to assess.

Objectives. This thesis develops a boundary-first framework for reasoning about reliability in event-driven processes. The aim is to clarify how explicit correctness boundaries shape process-handling choices in systems where asynchronous event processing performs correctness-relevant work.

Methods. The thesis follows a design science research approach. In design-science terms, the framework is the central artifact, while the guide developed later gives that framework a more concrete form. The framework is examined through a controlled empirical suite. Three working boundary classes are studied: deadline-constrained, state-oriented, and required-effect. These are examined across three process-handling models: transient/immediate, retained/immediate, and retained/deferred.

Results. Within the controlled scenarios examined here, the results show that the value of replay, retention, immediate handling, and deferred settlement differs according to the correctness boundary being protected. In deadline-constrained work, recovery matters only while unsettled work can still be repaid before expiry. In state-oriented work, replay matters when missed state remains relevant, while later state may also overtake earlier misses. In required-effect work, replay can recover downstream handling gaps after emission, but not source omission, and deferred settlement reduces one kind of immediate pressure by moving part of the burden into later reconciliation.

Conclusions. The thesis contributes a process-level framework for comparing handling choices under explicit correctness requirements. Its value lies in making the reasoning inspectable: what condition is protected, what disturbance threatens it, which handling model is chosen, and what trade-off follows. The guide gives this reasoning a more concrete form, while the contribution remains the framework itself and the disciplined comparison it makes possible.

Keywords: event-driven processes, reliability, design science research, correctness boundaries, process-handling models

Contents

Abstract	i
1 Introduction	1
1.1 Background and motivation	1
1.2 Research questions	2
1.3 Outline of the thesis	2
2 Related Work	3
2.1 Event-driven work as case-level obligation	3
2.2 Correctness boundaries, mechanisms, and process-handling models . .	4
2.3 Three working boundary classes	5
2.4 Disturbance as an analytical lens	6
2.5 Positioning and contribution	6
3 Method	9
3.1 Research design	9
3.2 Experimental environment	9
3.3 Scenario and run structure	10
3.4 Operationalising the correctness boundaries	11
3.5 Metrics and interpretation	12
3.6 Reporting discipline and scope	13
4 Results and Analysis	15
4.1 Deadline-constrained results	15
4.2 State-oriented results	18
4.3 Required-effect results	19
5 Discussion	23
5.1 Answer to RQ1: which conditions define correctness across different event-driven processes?	23
5.2 Answer to RQ2: how do process-handling models shape preservation, replay, repayment, and settlement under disturbance?	24
5.3 Answer to RQ3: correctness boundaries and process-handling choice .	25
5.3.1 Boundary-first design guide	25
5.3.2 Producer omission and producer-integrity controls	26
5.3.3 Illustrative design examples	26
5.3.4 Scope and realism	27
5.4 Relation to established work	28

5.5	Limitations and claim boundaries	29
6	Conclusions and Future Work	31
6.1	Future Work	31
A	Supplementary method figure	33
B	Supplementary summary table	35
C	Boundary-first design guide	37
D	Supplementary method tables	41
	References	47

List of Figures

4.1	Deadline outcome burden across the canonical scenario spine. The bars show the median number of expired cases across ten repetitions for baseline, backlog shock, moderate degradation, and high degradation under the three process-handling models.	16
4.2	Runtime resolution activity and final deadline burden under retained/deferred handling. Panel (a) shows completions and expiries per 250 ms bin, while panel (b) shows the corresponding final expired-case count.	17
4.3	Cumulative accounting for the backlog-shock case. The figure shows produced cases, in-time completions, and expiries over the run.	17
4.4	State-oriented adequacy under backlog shock and forward resume. The figure compares latest-state attainment and outage-exposed seen fraction for transient/immediate and retained/immediate handling.	19
4.5	Source omission boundary measured as received events. The figure compares handling gap and source omission across the three process-handling models, with the intended case count shown as a reference.	20
4.6	Duplicate-pressure containment. The figure shows duplicate downstream side-effects under duplicate pressure for immediate and deferred handling.	21
A.1	Experimental execution pipeline from profile input to aggregated outputs.	33

List of Tables

3.1	Frozen empirical suite manifest.	11
3.2	Operationalisation of correctness boundaries.	12
3.3	Boundary-aligned metric groups.	13
5.1	Compact use of the boundary-first guide.	25
B.1	Design scenario summary	36
C.1	Boundary-first guide for process-handling choice.	39
D.1	Operational definitions of reported metrics.	42
D.2	Scenario and parameter specification.	43
D.3	Disturbance-to-channel map.	45
D.4	Evaluation controls and scope protection.	46

1.1 Background and motivation

Event-driven systems are often praised for their scalability, flexibility, and loose coupling, for good reason. Yet those same qualities can also make correctness harder to see. Work is distributed across asynchronous components, and a case may continue to move through the system even as delay, loss, duplication, or omission changes what can still be achieved. The result is a practical difficulty rather than only a technical one: before a mechanism can be judged, the process condition it is meant to protect has to be made clear.

Part of the difficulty is that reliability in event-driven processes is often discussed through the language of mechanisms: replay, retention, exactly-once handling, idempotence, or eventual consistency. These mechanisms matter, but they do not answer the prior question by themselves: what must still hold for a case to count as acceptable? When that condition remains implicit, mechanism language can end up carrying more design meaning than it can bear on its own.

This problem is visible in both research and practice. Studies of event-driven and microservice systems report persistent difficulty around event schemas, event-flow auditing, ordering constraints, consistency, and transactional messaging support [1, 2]. Stream-processing work similarly treats correctness, latency, and cost as interacting concerns rather than as properties that can be judged in isolation [3]. These lines of work point to the same underlying need: mechanisms have to be interpreted in relation to the condition they are meant to protect.

A correctness boundary states what must hold for a case to count as acceptable. The corresponding mechanism concerns how the system attempts to meet that requirement. Keeping those levels distinct makes reliability questions easier to interpret and easier to test. It also fits a longer distributed-systems tradition in which the adequacy of a mechanism cannot be judged before the condition it is meant to protect has been stated [4, 5].

From that starting point, three categories of correctness demand come into view. Some work must reach a valid outcome within a limited window. Some depends on whether the visible state remains acceptable despite loss or delay. Some depends on whether one downstream action happens reliably and without harmful duplication. These are not offered as a complete account of distributed correctness. They are simply a useful way of making different obligations visible, and of comparing more clearly what kind of handling each one calls for [4, 6, 7].

The thesis argues that the appropriate handling model depends on what must

be protected, and on the cost and coordination introduced by protecting it in one way rather than another. That view is consistent with end-to-end reasoning and with broader work showing that coordination, recovery, and consistency costs are not evenly distributed across different kinds of work [5, 8, 9].

The thesis is design-oriented. Its central contribution is a boundary-first framework for judging process-handling models in event-driven processes, with the guide developed later in the thesis giving that framework a more concrete form. The empirical work examines the framework through a controlled empirical suite. Its role is to show how different handling models behave under disturbance and to make selected implications of the framework visible under defined conditions.

1.2 Research questions

The thesis is guided by three research questions.

RQ1. Which conditions define correctness across different event-driven processes?

RQ2. How do process-handling models shape preservation, replay, repayment, and settlement under disturbance?

RQ3. How can explicit correctness boundaries guide process-handling choices in event-driven design?

1.3 Outline of the thesis

Chapter 2 situates the thesis in relation to the literature most relevant to its argument and develops the conceptual basis for what follows. Chapter 3 explains the research design, the interpretive metrics, the internal checking and freeze discipline, and the study's scope limits. Chapter 4 presents the empirical results across the three correctness categories. Chapter 5 discusses what those results mean in relation to the research questions, the design guidance developed from them, and the established literature. Chapter 6 closes the thesis and points to future work.

This chapter positions the thesis within the bodies of work most directly relevant to its argument. The goal is not to survey event-driven systems exhaustively, but to identify the strands of distributed-systems and event-driven literature that matter most for how correctness, recovery, and process-handling choices are understood.

The study sits between established distributed-systems research and practical event-driven design. Prior work has given rich accounts of publish-subscribe decoupling, end-to-end reasoning, fault recovery, tail behavior, replication, stream processing, and compensating action [3, 5, 6, 7, 10, 11, 12, 13]. These works provide the background against which the thesis should be read. They show that reliability is never only a matter of moving messages. It is also a matter of what a system is trying to preserve, how that preservation is attempted, and what costs follow from the chosen form of handling.

The gap addressed here is therefore a gap in connection rather than a lack of mechanisms. Event-driven systems already offer many ways to publish, retain, replay, process, coordinate, and settle work. What is less settled is how those means should be judged when a designer must decide what kind of correctness is actually at stake in a concrete flow. This matters all the more as event-streaming and stream-processing infrastructure have become more common and prominent in both research and practice [13]. The more readily such support is available, the easier it becomes to reason from available capability while leaving the foundational question unstated: what, in concrete terms, must still hold for the process to count as correct?

In practical system discussions, the same vocabulary is often used for quite different obligations. Replay, retention, immediate handling, idempotence, eventual consistency, and compensation all matter, but they do not matter in the same way for every flow. Work on event-management challenges and dataflow trade-offs suggests that correctness concerns are often more varied than the language around them admits [1, 3, 14]. This thesis starts from that variation.

2.1 Event-driven work as case-level obligation

The unit of analysis in this thesis is not the event in isolation. It is the case or process whose acceptability may depend on what the event causes, preserves, updates, or triggers. An event is important because it participates in some larger piece of work. It may report a change in state, trigger a downstream action, carry work forward, or make recovery possible after a handling gap. The same technical treatment of an event can therefore be adequate in one flow and inadequate in another.

This is why event delivery alone is too narrow a view of reliability. A delivered event may arrive too late to help. A retained event may be replayable after the point at which its result can still count as correct. A missed state update may be harmless if later state legitimately overtakes it. A downstream action may still be wrong if it happens twice, even when every individual message was processed according to the mechanism's local rules. In each case, the relevant question is not only whether the event moved through the system. The question is whether the process still satisfies the condition that makes it acceptable.

This case-level view extends an end-to-end style of reasoning into event-driven process design. Saltzer, Reed, and Clark show that a lower-level mechanism may be useful and still be insufficient if the final requirement must be checked at the application boundary [5]. The same discipline applies here. Replay, retention, idempotence, and deferred settlement are not strong or weak in themselves. They are adequate or inadequate in relation to the condition the process must protect. The contribution of this thesis is to express that discipline in process-facing terms for event-driven reliability.

2.2 Correctness boundaries, mechanisms, and process-handling models

The thesis uses three connected ideas. A correctness boundary states what must hold for a case to count as acceptable. A mechanism is a means used to protect or fulfil that condition. A process-handling model is the way such means are combined in a particular event-driven process.

Put more simply: correctness boundaries state what must hold. Mechanisms provide the means available. Process-handling models combine those means in different ways. The design question is which handling model is appropriate to the condition a process must protect.

This distinction matters because mechanisms do not have a fixed reliability meaning on their own. Replay, for example, means that emitted events can be processed again. That does not by itself say whether replay protects the process. Replay can recover emitted work after a downstream handling gap, but it cannot recover an event that was never emitted. It can preserve work for later completion, but that may not help if the work settles after a deadline has passed. Its value depends on the correctness boundary it is used to protect and on the handling model in which it operates.

The same is true of retention, immediate handling, deferred settlement, idempotence, producer-side emission safeguards, and reconciliation. Each matters only in relation to the condition it is used to protect. A retained stream matters when preserved work remains relevant. Immediate handling matters when delay or loss would invalidate the case. Deferred settlement matters when later reconciliation is acceptable and when reducing harmful duplication is worth the cost it introduces. Producer-side safeguards matter when the event must enter the flow before any downstream mechanism can act on it.

A process-handling model is therefore not a single mechanism. It is the arrange-

ment through which the process attempts to keep the boundary true. In the empirical part of the thesis, this idea is represented by three deliberately simple handling models. In transient/immediate handling, the process depends on work being handled as it arrives, or on missed work not remaining correctness-relevant. In retained/immediate handling, emitted work remains available for replay after a handling gap. In retained/deferred handling, emitted work is preserved first and settled later, often trading immediate completion for duplicate containment or controlled reconciliation.

These handling models are simplified, but they are useful because they make the design question visible. They show that a handling choice has to be judged against the condition the process must protect.

2.3 Three working boundary classes

To make the framework concrete, the thesis focuses on deadline-constrained, state-oriented, and required-effect work. This focus reflects a basic feature of event-driven process work: what counts as correct often depends on whether a result is still valid in time, whether the state reached by the process is acceptable, or whether a required downstream action has been carried through correctly. These conditions do not exhaust the correctness concerns that can arise in event-driven processes, but they are central to ordinary event-driven process design. They ground the comparison by making clear what each handling model is being asked to protect: timely completion, acceptable state, or one downstream action happening reliably and without harmful duplication.

In deadline-constrained work, the protected condition is timely completion. A case must reach a valid outcome while that outcome still counts. This makes delay and repayment central. Preserved work is not enough if it settles after the validity window has closed. For this kind of work, the decisive question is whether recovery can still arrive in time.

In state-oriented work, the protected condition is an acceptable visible or resulting state. A missed update is not always a lasting correctness failure. Later state may overtake the missed information and restore the condition that matters. At the same time, final adequacy and outage-time exposure are not identical. A process may end in an acceptable state while still exposing incomplete or misleading state during the disturbance.

In required-effect work, the protected condition is whether one downstream action happens reliably and without harmful duplication. This boundary makes it necessary to separate producer integrity, downstream recovery, duplicate control, and settlement cost. If a valid event has been emitted, replay can recover a downstream handling gap. If the event was never emitted, downstream replay has no object to act on. Producer-side safeguards are therefore important, but they protect entry into the event flow rather than forming a fourth process-handling model. If repeated triggers or redelivery can cause repeated effects, idempotence or deferred settlement may be needed, but those mechanisms address a different risk and may shift cost into reconciliation or delayed settlement.

These three boundary classes make the central point visible. Mechanisms should not be judged in isolation. Their value depends on the condition they are used to

protect. A retained event, a replay path, an immediate consumer, or a deferred settlement step has different significance depending on whether the process is trying to finish in time, preserve an acceptable state, or ensure that one downstream action happens reliably and without harmful duplication.

2.4 Disturbance as an analytical lens

The empirical work in the thesis focuses on disturbance because smooth event flow often hides the distinctions the framework is meant to reveal. When events are emitted, delivered, handled, and settled without delay or loss, different handling models can appear equivalent. The process succeeds, and the reason for success is not very informative. The harder question is what happens when delay, loss, duplicate pressure, omission, or backlog interrupts that ordinary path.

Disturbance is therefore not an exceptional corner of the argument. It is the lens through which the consequences of a handling model become visible. Under disturbance, the system reveals whether work was merely delivered or actually protected. It shows whether retained work can still settle in time, whether missed state still matters, whether a downstream handling gap remains replayable, and whether duplicate containment shifts cost into later reconciliation.

Replay again illustrates the point. In a smooth run, replay may never be called upon, so its practical value remains latent. Under a downstream handling gap, retained replay can recover work that transient handling loses. Under source omission, the same replay capability has nothing to act on. Under a deadline boundary, replay may preserve work that still arrives too late to count. Under a state boundary, replay may be unnecessary if later state has already restored the condition that matters. Replay is therefore not simply good or bad. Its value depends on the boundary being protected and on the handling model in which it operates.

This also explains why the thesis gives attention to repayment and settlement, not only to final success. A case can eventually complete while still exposing an unacceptable delay. A process can end in an adequate state while still exposing incomplete or misleading state during the outage. A deferred step can restrain harmful duplicates while adding reconciliation work and delayed settlement. Disturbance makes these differences observable. It turns mechanism choice into a question of what was protected, what was merely postponed, and what cost was moved elsewhere.

2.5 Positioning and contribution

The contribution is best understood as restrained and layered. At its centre is a boundary-first way of reasoning about event-driven reliability. The thesis starts by asking what must hold for a case to count as acceptable, then examines how different process-handling models protect that condition under disturbance. It does not propose a new reliability mechanism, stream-processing model, benchmark, or complete taxonomy. Its purpose is to clarify how existing mechanisms become relevant, or insufficient, once the protected condition is made explicit.

This places the thesis close in spirit to work on coordination avoidance and CALM, though it does not adopt their formal criteria. Those lines of work show

that the need for coordination depends on the property being protected, especially application-level invariants and whether a computation can proceed without coordination [8, 9]. The present thesis makes an analogous process-level move for event-driven reliability. It asks which condition the process must protect before judging whether replay, retention, immediate handling, deferred settlement, idempotence, or producer-side safeguards are warranted.

The empirical part of the thesis is positioned as a controlled comparison rather than as a production benchmark. Within the controlled scenarios examined here, the instrument supports the narrower claim that the value of replay, retention, immediate handling, and deferred settlement differs according to the correctness boundary being protected. The reported metrics are used to show how each handling model affects the condition at stake in a given process: whether work misses its deadline, state is preserved or overtaken, an effect is duplicated, or recovery is shifted into later settlement.

The thesis should therefore be read as a complementary clarification within established work on correctness, recovery, stream processing, and coordination. Current research continues to show difficulty around event schemas, event-flow auditing, ordering constraints, recovery behaviour, and transactional messaging support in event-driven and microservice systems [1, 2, 15, 16]. Against that setting, the thesis does not claim that one handling model is generally superior. It argues that the value of a model depends on what the event-driven process must protect. Once the boundary is explicit, questions about replay, retention, immediate handling, deferred settlement, producer safeguards, and recovery become easier to state and easier to judge.

3.1 Research design

The thesis follows a design science research approach: it develops a boundary-first framework and examines it through a controlled empirical suite [17, 18]. In design-science terms, the framework is the central artifact, while the guide developed later gives that framework a more concrete form. The guide is not treated as a separately validated design method. The framework makes three questions explicit for an event-driven process: what correctness requires, what kind of disturbance threatens that requirement, and which handling model is appropriate to the condition the process must protect.

The empirical suite is the research instrument used to examine that framework. It does not try to reproduce full system complexity. It deliberately abstracts from that complexity so the analysis can focus on the patterns most relevant to the thesis: preservation, replay, repayment, and settlement under different correctness boundaries.

The method follows the structure developed in Chapter 2. Correctness boundaries define what must hold. Process-handling models define how event-driven work is handled, preserved, replayed, or settled. Disturbance makes the consequences of those choices visible. Chapter 3 describes how those elements are represented in the research instrument.

3.2 Experimental environment

The empirical suite is implemented as a small Docker-based event-processing environment. Docker is used to keep the runtime arrangement repeatable and to separate the main roles in the experiment: event production, event transport, consumer handling, retained replay, deferred settlement, and reporting [19]. The environment is compact by design, because the study depends on comparing selected handling behaviours clearly.

Redis provides the event substrate. Redis Pub/Sub is used for transient publication, where events are delivered to active subscribers but are not retained for later replay [20]. Redis Streams is used for retained event handling, where emitted events remain available after initial delivery and can therefore support replay [21]. This combination gives the suite the contrast needed by the thesis: handling that depends on arrival-time processing, handling that can recover emitted work through replay, and handling that separates preservation from later settlement.

The consumers implement the handling behaviour examined in the suite. In immediate handling, work is attempted as events arrive or are replayed. In deferred handling, emitted work is preserved first and resolved through a later settlement step. Reporting scripts collect run outputs and derive the measures used in the analysis: attainment, expiry, unresolved and doomed burden, replay recovery, state adequacy, outage-time exposure, duplicate effects, reconciliation work, and settlement delay.

The execution pipeline is included in Appendix A as Figure A.1. It shows how profile input, Docker execution, Redis services, and batch aggregation connect in the reported suite.

The same principle applies to disturbance. The suite does not model every concrete reason why event handling might slow down or fail. In an ordinary event-driven process, a consumer may fall behind because the work it performs takes longer than expected, for example while waiting on a database operation, a downstream service, or another application-level side effect. The suite abstracts those implementation details into the effects that matter here: processing takes longer, work accumulates, emitted work is missed, or repeated triggers bear on the same downstream obligation. Source omission is treated separately, because there the event never enters the flow and replay has nothing to recover.

3.3 Scenario and run structure

The suite is organised around the three correctness boundaries developed in Chapter 2. Each module keeps the scenario structure small enough to interpret, while varying the disturbance most relevant to that boundary.

The deadline-constrained module examines timely completion. It uses four scenario families: baseline, moderate degradation, high degradation, and backlog shock. Baseline provides the reference condition. Moderate and high degradation represent sustained consumer-side pressure at two intensities. Backlog shock represents a sharper disturbance in which work accumulates and must later be repaid. These scenarios are compared across the three process-handling models: transient/immediate, retained/immediate, and retained/deferred.

The state-oriented module examines whether missed or delayed updates still matter to the resulting or visible state. The comparison is smaller than in the deadline module because the question is narrower: whether replay contributes to final correctness when state updates are missed or delayed. In the continuous-update setting examined here, later state may overtake an earlier missed update, making simple transient handling sufficient at lower cost. The point is not that replay is irrelevant for state-oriented work. Where updates are infrequent, or where a missed update can leave state stale for a long period, retained replay may still be appropriate.

The required-effect module examines whether one downstream action happens reliably and without harmful duplication. It uses three disturbance patterns. A handling gap represents emitted work that was missed downstream. Duplicate pressure represents repeated triggers bearing on the same downstream obligation. Source omission represents the case where the event never enters the flow. These scenarios make it possible to distinguish replayable downstream loss from unrecoverable source omission, and recovery from duplicate containment.

Each reported run follows the same basic sequence. The producer emits the configured case or event workload. The selected handling model processes the workload under the configured disturbance. The suite records both endpoint outcomes and runtime traces. The reporting step then derives the measures used in the analysis. The final reporting suite uses fixed scenario definitions, fixed extraction logic, fixed plotting conventions, and ten repeated runs for each reported scenario cell.

Table 3.1: Frozen empirical suite manifest.

Module	Scenario cells	Repetitions per cell	Completed runs	Role in the argument
Deadline-constrained	12	10	120	Examines repayment, expiry burden, and runtime-versus-endpoint contrast.
Required-effect	18	10	180	Examines replayable handling gaps, source omission, duplicate containment, and settlement cost.
State-oriented	4	10	40	Examines latest-state adequacy, outage-time exposure, and forward resumption.
Total	34	10	340	Frozen reporting suite used for the thesis comparison.

The exact parameter values are reported in Appendix D, Table D.2. The prose focuses on the role each condition plays in the comparison: moderate degradation is sustained pressure, high degradation is heavier sustained pressure, and backlog shock is concentrated pressure followed by repayment opportunity. The analysis depends on those comparative roles rather than on treating any single parameter value as a general threshold.

3.4 Operationalising the correctness boundaries

The conceptual distinctions in Chapter 2 are made observable through measures tied to each boundary class. The measures are chosen because they expose the differences the thesis needs to compare.

Table 3.2: Operationalisation of correctness boundaries.

Boundary class	Empirical question	Main observations
Deadline-constrained	Can affected cases still reach a valid outcome before the window closes?	Attained and expired outcomes, unresolved and doomed burden, peak and integrated burden, repayability, and runtime traces.
State-oriented	Does the resulting or visible state remain acceptable despite missed or delayed updates?	Final state adequacy, outage-time exposure, and later state overtaking missed updates.
Required-effect	Does one downstream action happen reliably and without harmful duplication?	Unattained effects, handling-gap recovery, source omission, duplicate side-effects, correction rewrites, reconciliation passes, and settlement delay.

For deadline-constrained work, eventual completion is not enough. Completion must still arrive while it counts. The relevant question is whether unsettled work can still be repaid before expiry. This is why the analysis reads endpoint outcomes together with runtime traces of unresolved and doomed burden.

For state-oriented work, the relevant question is whether missed or delayed updates still matter to the state that must be preserved. A missed update may leave the final state wrong, or it may be overtaken by later state. The analysis therefore separates final adequacy from outage-time exposure.

For required-effect work, the analysis separates recovery from duplicate containment. Replay can recover a downstream handling gap only if the event was emitted. It cannot recover source omission. At the same time, a model may recover missed work while still producing harmful repeated effects, or it may reduce duplicate effects while shifting cost into later settlement.

3.5 Metrics and interpretation

The metrics are chosen to make the protected boundary visible, not to compare platform throughput or latency in the abstract. This follows a broader stream-processing concern: correctness, latency, and cost interact, and an output may have different value depending on whether it arrives in time, is later corrected, or is superseded by newer state [3]. The analysis therefore connects final outcomes with the path by which those outcomes were reached. Endpoint measures show whether the relevant correctness condition was ultimately met. Runtime and settlement measures show how the result came about: whether work accumulated, whether recovery still arrived in time, whether missed work remained replayable, whether state was visible during a disturbance, and whether duplicate containment shifted cost into later reconciliation. Table 3.3 summarises the metric groups used in the analysis. The full operational definitions are provided in Appendix D, Table D.1.

Table 3.3: Boundary-aligned metric groups.

Boundary class	Main empirical reading	Supporting measures
Deadline-constrained	Whether cases can still reach a valid outcome before the deadline closes.	Deadline miss rate, completion-in-time rate, unresolved burden, doomed burden, repayment pattern, and resolution pulse.
State-oriented	Whether final state adequacy and outage-time exposure tell the same story.	Latest-state attainment rate and outage-exposed seen fraction.
Required-effect	Whether the downstream action happens reliably and without harmful duplication.	Required-effect attainment, received-event count, duplicate side-effects, correction rewrites, reconciliation passes, and settlement delay.
Cross-cutting interpretation	Whether the final outcome alone hides the path by which it was reached.	Endpoint outcomes read together with runtime traces, replay signals, and settlement-cost measures.

The main interpretive point is that final outcomes are not enough on their own. A process that eventually completes may still fail a deadline boundary. A process that ends in an adequate state may still expose incomplete or misleading state during disturbance. A process that restrains duplicates may do so by introducing reconciliation work and delay. For that reason, the results are read through both endpoint outcomes and runtime traces. This emphasis on runtime meaning is consistent with recovery-oriented thinking and with recent benchmarking work on fault recovery behaviour [11, 12, 15, 16].

3.6 Reporting discipline and scope

The reported results come from the frozen suite described above: fixed scenario definitions, fixed output extraction, fixed plotting conventions, and fixed reporting measures. Preliminary runs were used to check that the main comparative patterns remained stable enough to support the argument. The thesis therefore treats the empirical material as a controlled comparison across defined conditions.

Within this abstraction, the suite supports comparative interpretation. The measured values are read as evidence about the relative behaviour of the handling models under the defined disturbances. They are not treated as transferable performance values for external systems or workloads.

Within the controlled scenarios examined here, the instrument supports the narrower claim that the value of replay, retention, immediate handling, and deferred settlement differs according to the correctness boundary being protected.

The results are organised around three empirical sections that examine the framework under different correctness boundaries. Each section makes a different distinction visible: deadline-constrained results focus on repayment before expiry, state-oriented results on final adequacy and outage-time exposure, and required-effect results on emitted handling gaps, source omission, duplicate pressure, and settlement cost.

Across the chapter, outcomes are interpreted against the boundary they bear on. A recovered backlog, a converged final state, or a reduced duplicate count may look favourable, but the relevant question is whether it still satisfies the condition the process is meant to protect.

4.1 Deadline-constrained results

The central question in this section is whether cases affected by disturbance can still be brought to a valid outcome before the deadline passes. From that standpoint, backlog becomes analytically important when it threatens timely completion. The key issue is not simply whether work remains available, but whether unsettled work can still be repaid before the validity window closes [3, 12].

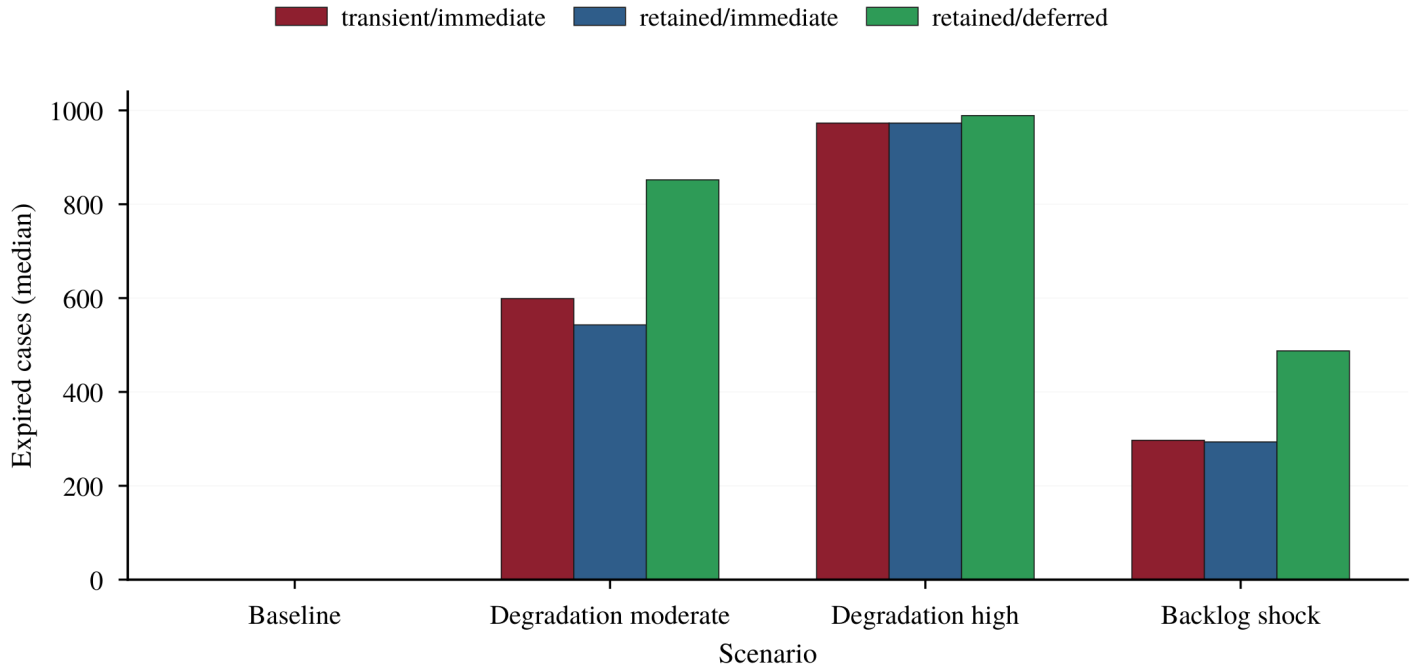


Figure 4.1: Deadline outcome burden across the canonical scenario spine. The bars show the median number of expired cases across ten repetitions for baseline, backlog shock, moderate degradation, and high degradation under the three process-handling models.

The endpoint view first shows that deadline harm is not simply a function of how dramatic the disturbance appears at runtime. Baseline produces no deadline burden, while backlog shock and sustained degradation produce different levels of expiry. The important distinction is that a short, sharp disruption can be less damaging than a sustained slowdown if enough time remains for repayment. For deadline-constrained work, the endpoint result therefore points back to the boundary itself: preserved work matters only if it can still be settled before the deadline closes.

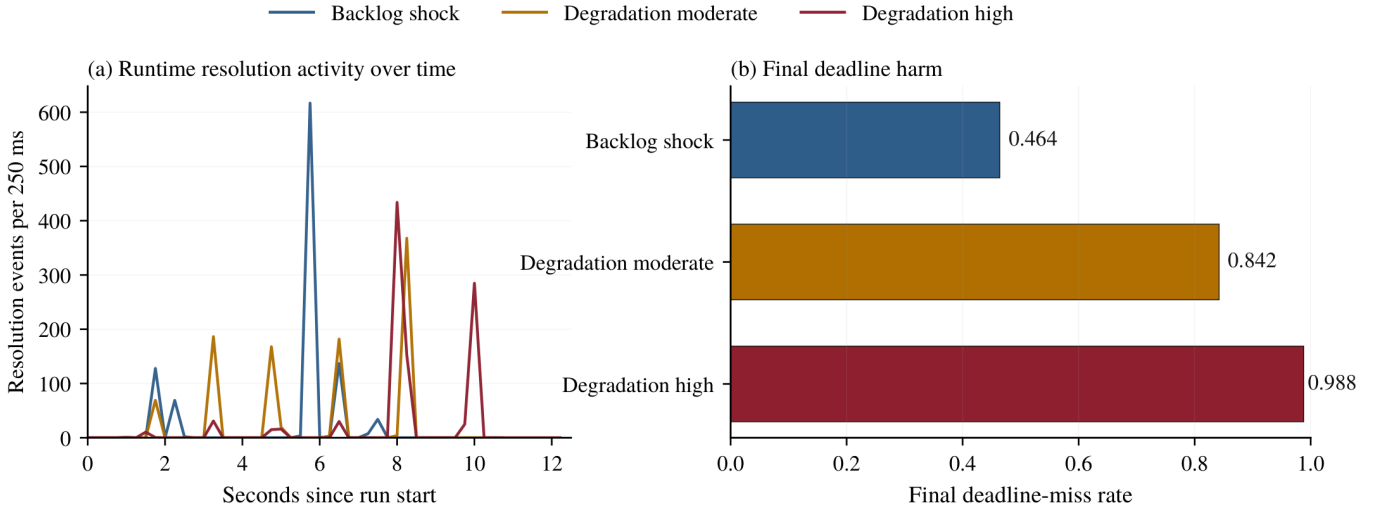


Figure 4.2: Runtime resolution activity and final deadline burden under retained/deferred handling. Panel (a) shows completions and expiries per 250 ms bin, while panel (b) shows the corresponding final expired-case count.

The runtime view sharpens the same distinction. Under retained/deferred handling, backlog shock produces the most visible burst of resolution activity, but not the greatest final deadline harm. Moderate and high degradation are less visually dramatic, yet they leave more cases beyond the deadline because they reduce the time available for repayment throughout the run. The figure therefore shows why runtime activity and endpoint damage have to be interpreted together.

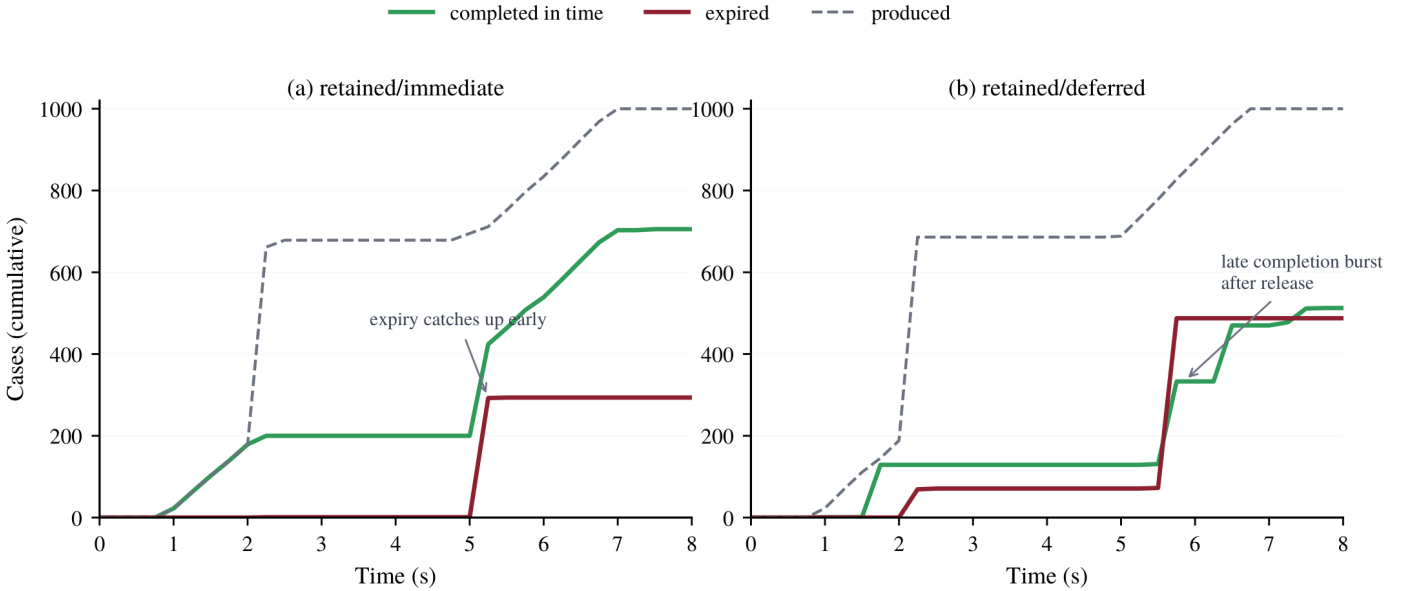


Figure 4.3: Cumulative accounting for the backlog-shock case. The figure shows produced cases, in-time completions, and expiries over the run.

The cumulative view gives a more concrete accounting of the backlog-shock case.

Produced cases, completions in time, and expiries are shown over the run. The retained/deferred trace includes a visible late completion burst after release, but much of the deadline damage has already occurred by then. The point is not that recovery activity is absent. It is that recovery activity matters for this boundary only if it arrives while the outcome can still count as correct.

Taken together, the figures show why deadline-constrained work is governed less by peak pressure alone than by whether unsettled work can still be repaid before expiry. Retention and replay can keep work available, but availability is not the same as timely settlement. The design implication is that deadline reliability is not only a question of whether work can eventually be recovered. It is a question of whether the remaining time still makes recovery meaningful. The handling model for deadline-constrained work should therefore be judged by whether recovery can still arrive in time for correctness to hold.

4.2 State-oriented results

This section examines a different correctness question: not whether work is completed within a deadline window, but whether the state represented by the process remains acceptable despite missed or delayed updates. From that standpoint, replay is not valuable in itself. Its value depends on whether a missed update remains relevant to the state the process is required to preserve [6, 22].

Two measures are used to separate these cases. Latest-state attainment measures whether the process reaches the expected latest version for the case. Outage-exposed seen fraction measures whether updates that occur during the disturbance are observed by the handling model. The first measure concerns final adequacy. The second concerns outage-time exposure: whether the disturbance hid updates that were relevant while the outage was in effect. The distinction matters because a process may end in an acceptable state even when some intermediate state was missed, while in other cases that missed state may remain relevant.

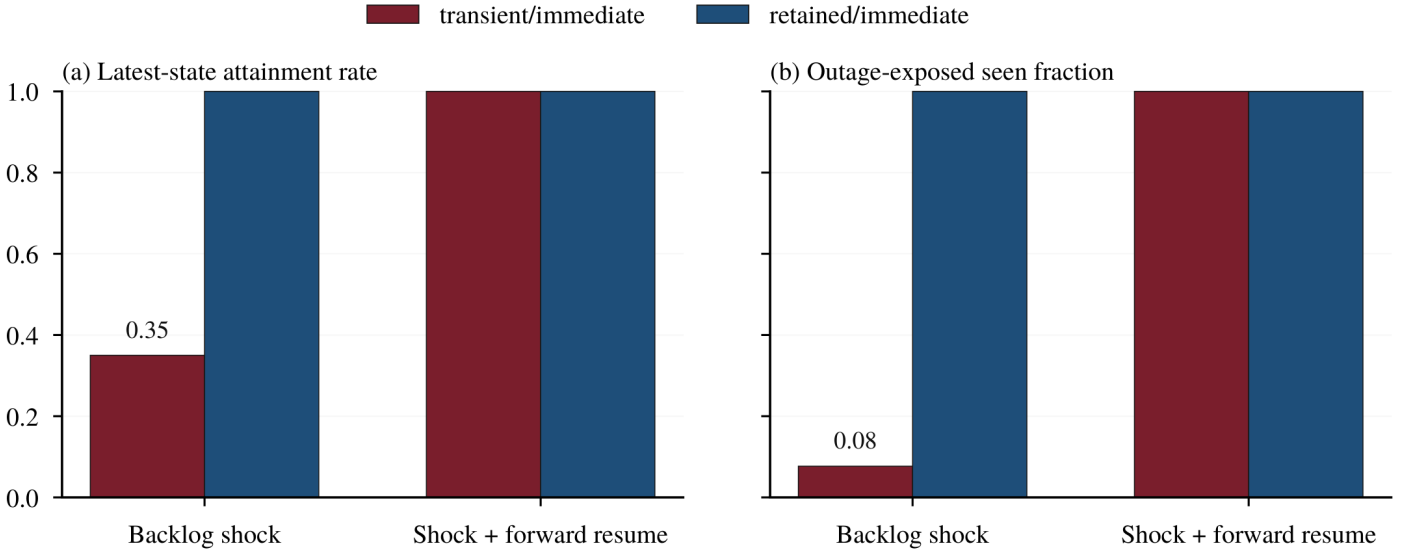


Figure 4.4: State-oriented adequacy under backlog shock and forward resume. The figure compares latest-state attainment and outage-exposed seen fraction for transient/immediate and retained/immediate handling.

The figure shows why replay relevance is conditional on the state boundary. Under backlog shock, transient handling loses state-relevant updates, while retained handling preserves them for replay. Under forward resume, later updates overtake the missed information, and both models reach full latest-state attainment. The same missed-update pattern therefore has different meaning depending on the state the process is required to preserve. Replay matters when it restores state that still counts. It matters less when newer state has already made the missed update obsolete.

Taken together, the state-oriented results suggest a restrained design implication. In the continuous-update setting examined here, later state may overtake an earlier missed update, making transient handling sufficient and sometimes the lower-cost choice, because replay may reintroduce state that has already lost its relevance. Where updates are infrequent, or where a missed update can leave state stale for a long period, retained replay may still be appropriate. The broader point is that state-oriented correctness is not measured simply by event deficit or replay completion. It depends on whether the missed update still belongs to the state the process is required to preserve. The handling model for such work should therefore be judged by whether replay contributes to that state condition.

4.3 Required-effect results

This section examines cases in which correctness depends on whether one downstream action happens reliably and without harmful duplication. The central issue here is not only whether an event can be processed, but where the relevant failure occurs. A downstream handling gap can remain recoverable if the event has entered the flow. Source omission is different because some logical cases never produce an

event for replay or retained handling to act on. Duplicate pressure raises a further question: whether repeated triggers lead to harmful repeated effects, and what cost is introduced when the handling model contains those effects [7, 14].

The first required-effect distinction concerns where the omission occurs. Once an event has been emitted, a downstream handling miss still lies within the reach of replay. Retained handling can recover work that transient handling leaves unresolved. That recovery logic stops at the source boundary: when the event never enters the flow, downstream replay has no object to recover.

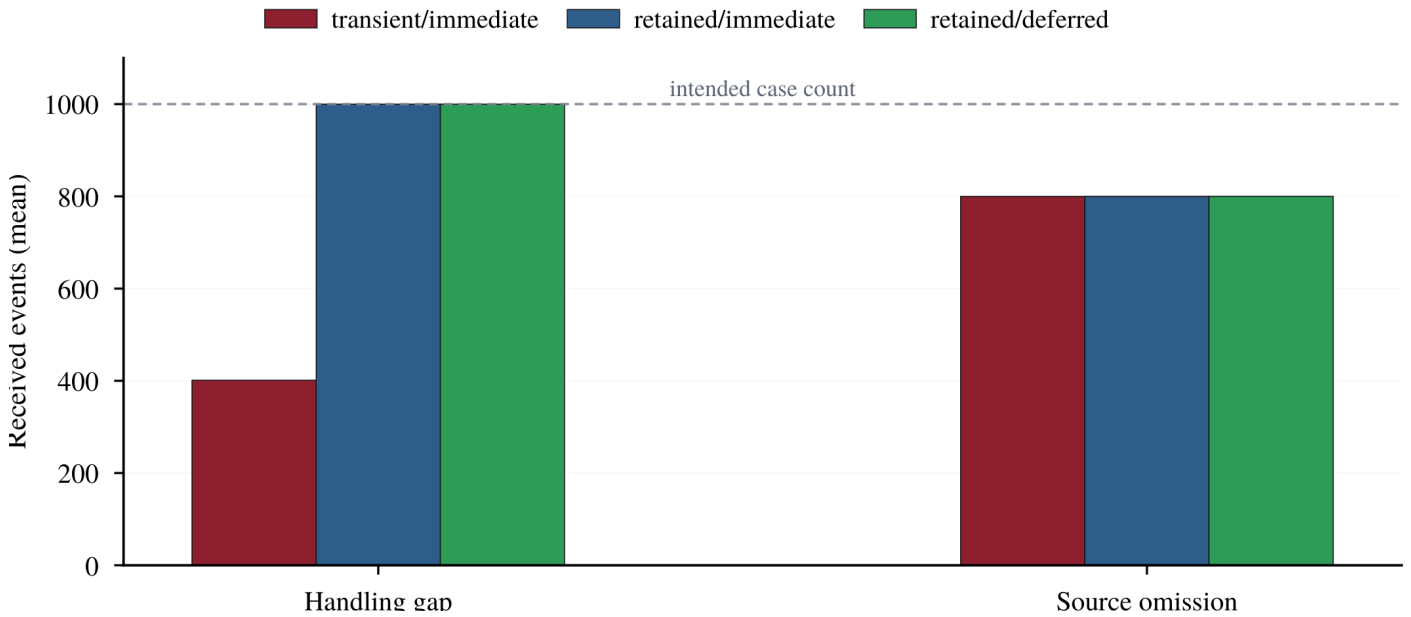


Figure 4.5: Source omission boundary measured as received events. The figure compares handling gap and source omission across the three process-handling models, with the intended case count shown as a reference.

Figure 4.5 is best read as a boundary check rather than as a surprising empirical result. Its value is that it marks where downstream recovery begins. In a handling gap, the event has entered the flow and retained handling can recover what transient handling misses. In source omission, some logical cases never produce an event, so the failure lies before replay can help. The figure therefore separates downstream recoverability from producer-side integrity, which is essential for required-effect work.

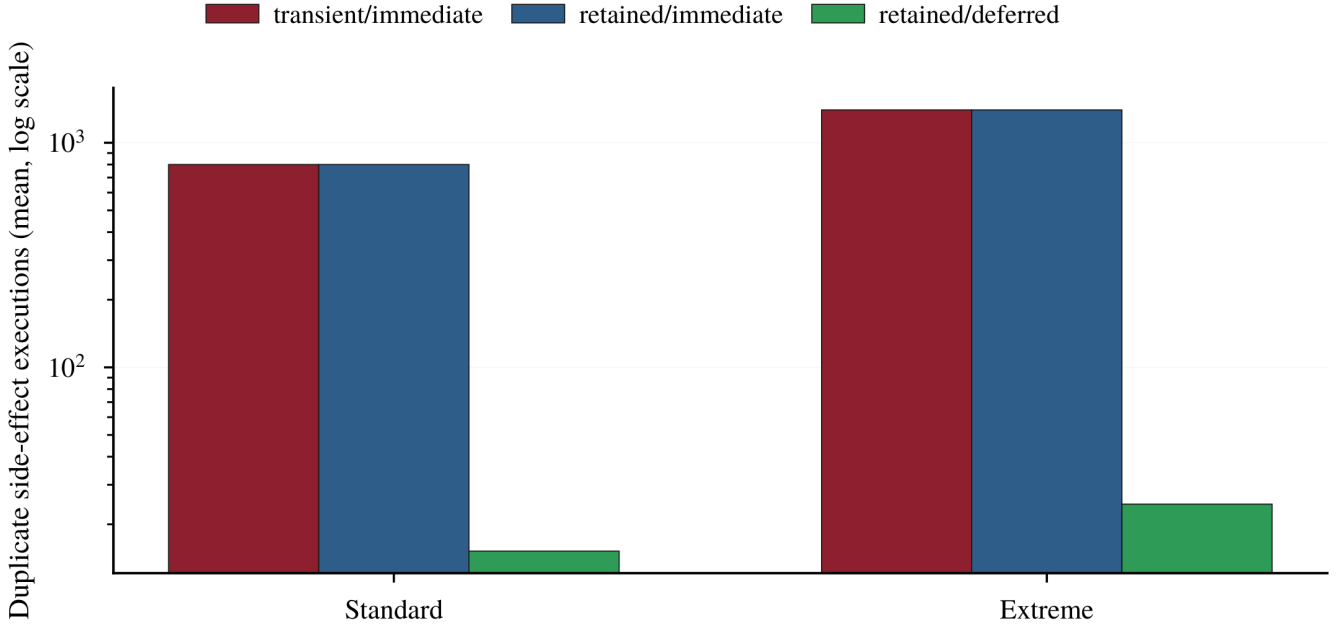


Figure 4.6: Duplicate-pressure containment. The figure shows duplicate downstream side-effects under duplicate pressure for immediate and deferred handling.

The duplicate-pressure figure addresses a different part of the required-effect boundary. The important contrast here is not transient versus retained, but immediate versus deferred handling. In the retained/deferred model used in the suite, repeated triggers are absorbed before final settlement, sharply reducing duplicate downstream execution. The figure therefore shows the containment side of required-effect handling, while the corresponding cost appears in later reconciliation and settlement work. The retained/deferred settlement measures reported in the supplementary material capture that cost through reconciliation passes, correction rewrites, and settlement delay.

Taken together, the required-effect results separate concerns that are easy to blur. Replay is relevant when a valid source event exists and downstream handling fails or is missed. It is irrelevant when source omission prevents the event from entering the flow in the first place. Duplicate containment raises a different issue again: deferred handling can reduce immediate downstream pressure, but it does so by moving part of the burden into later settlement. In the duplicate-pressure scenario examined here, this appears as fewer repeated downstream effects at the cost of reconciliation passes, correction rewrites, and settlement delay. A handling model for required-effect work should therefore be judged by both the location of the omission risk and the cost of postponing settlement.

The discussion begins from the thesis’s main contribution. The related-work chapter established the distributed-systems context most relevant to the argument. The task here is to interpret what the empirical sections add within that context. The central contribution lies in the framework developed for relating correctness conditions to process-handling choice. The empirical results support that argument by making selected distinctions visible within the controlled research setting, rather than by serving as a general benchmark of event-driven systems. Taken together, the thesis provides a clearer basis for assessing process-handling models in event-driven processes, and the discussion answers the three research questions in that order.

5.1 Answer to RQ1: which conditions define correctness across different event-driven processes?

In deadline-constrained processes, correctness is defined by whether delayed cases can still reach a valid outcome before the window closes. Delay matters because it can change the status of the outcome itself. A case may eventually complete and still fail if completion arrives after the point at which the result no longer counts. Time is therefore part of correctness, not merely a performance concern.

In state-oriented processes, correctness is defined by whether missed or delayed updates still matter to the state the process is required to preserve. A missed intermediate update is not necessarily a lasting failure, since later state may legitimately overtake it. The boundary is affected when the missed update remains the relevant state, or when the process exposes incomplete or misleading state during the disturbance. The central question is therefore state relevance, not event count.

In required-effect processes, correctness is defined by whether one downstream action happens reliably and without harmful duplication. The first distinction is where the failure occurs. Once an event has entered the flow, replay may recover a downstream handling gap. Before emission, there is no event for replay to recover. Repeated triggers create a separate risk: the action may happen more than once, so duplicate control and settlement cost become part of the boundary.

Taken together, these distinctions give correctness a process-level form. A process is not correct merely because an event was delivered, retained, replayed, or eventually settled. It is correct when the condition it protects still holds: the outcome arrived in time, the relevant state remained acceptable, or the required downstream action happened without being lost or harmfully repeated. This is the foundation

for judging the handling models discussed in the next section.

5.2 Answer to RQ2: how do process-handling models shape preservation, replay, repayment, and settlement under disturbance?

Process-handling models matter because they change which losses remain recoverable, which delays remain acceptable, and which costs are moved into later settlement. In the results presented here, three patterns stand out.

In deadline-sensitive work, the important question is not peak pressure alone, but whether unsettled work can still be repaid before expiry. The retained/deferred backlog-shock case makes this visible: the most dramatic runtime burst does not produce the greatest final deadline harm, because endpoint damage depends on whether accumulated work can still settle in time.

In state-oriented work, replay value depends on whether a missed update still matters to the state condition being protected. In the continuous-update setting examined here, later updates can overtake earlier missed state, making transient handling sufficient and sometimes the lower-cost choice. In other state-oriented settings, where missed updates remain relevant for longer, retained replay may still be the more appropriate model.

In required-effect work, downstream handling gaps and source omission are not the same kind of failure. Once an event has entered the flow, retained replay can recover missed downstream handling. If the event was never emitted, replay has no object to act on. Deferred handling adds a different trade-off: it can reduce immediate downstream pressure, but it does so by moving part of the burden into later settlement.

These are modest distinctions, but they sharpen the question of handling choice. They show that the same handling model can preserve what matters in one kind of work, arrive too late in another, or address the wrong failure altogether.

A further lesson is that recovery signs have to be read through the condition they are supposed to satisfy. A cleared backlog, a replayed event, a converged final state, or a lower duplicate count may all look favourable. None is decisive on its own. The relevant question is narrower: did the work still arrive in time, did the recovered state still matter, and did duplicate containment leave an acceptable settlement burden?

This is why the empirical suite follows both endpoint outcomes and the route by which those outcomes were reached. Endpoint results show where the process ended. Runtime and settlement traces show whether that endpoint still satisfied the condition at stake, or whether correctness had already been lost, superseded, or shifted into later work. The distinction is small in wording but important in design: a system can appear to recover while still failing the condition the process was meant to protect.

5.3 Answer to RQ3: how can explicit correctness boundaries guide process-handling choices in event-driven design?

The design implication follows from that comparison. A handling choice becomes easier to judge when it is considered at the level of a specific event-driven process, against the condition that process must protect. The guide developed below gives that judgment a more concrete form: state the boundary, identify the disturbance, and choose the lowest-sufficient handling model with its trade-off made clear.

5.3.1 Boundary-first design guide

The boundary-first guide is applied to one event-driven process at a time. A larger application may contain several such processes, but each process should first be judged against its own protected condition.

The compact form of the guide is shown in Table 5.1. Its purpose is to make the handling choice explicit and proportionate: the selected model should be sufficient for the boundary at stake, without treating replay, retention, or deferred settlement as inherently stronger in the abstract.

Table 5.1: Compact use of the boundary-first guide.

Step	Guiding question	Design output
Scope and state the boundary	Which event-driven process is being judged, and what must hold for its cases to count as acceptable?	A clearly scoped process and a primary boundary: deadline-constrained, state-oriented, or required-effect.
Identify the boundary-breaking disturbance	What kind of delay, loss, stale state, duplicate pressure, settlement delay, or source omission would make that condition false?	A concrete disturbance risk, including whether it lies inside downstream handling or before the event enters the flow.
Choose the handling model and evidence	Which lowest-sufficient handling model protects the boundary under that disturbance, and what residual risk remains?	A justified choice of transient/immediate, retained/immediate, or retained/deferred, together with the trade-off and the metric that would reveal failure.

The full guide is provided in Appendix C, Table C.1. It expands the three steps into the specific checks used throughout the thesis, including deadline sensitivity, state relevance, downstream recovery, duplicate harm, settlement tolerance, and source omission.

Source omission is included as a check rather than as a process-handling model. If the event may never enter the flow, the issue is not simply stronger downstream handling. Replay, retention, and deferred settlement become meaningful only after

the event exists in the flow. Producer-side integrity is therefore treated separately in the following subsection.

5.3.2 Producer omission and producer-integrity controls

An event can only be recovered downstream if it was emitted in the first place. Replay and retained handling can recover some forms of downstream loss, but they do nothing for a logical case that never produced an event. Where omission would materially affect the outcome, producer-side integrity controls should therefore be considered before downstream recovery is treated as sufficient.

A transactional outbox is one example. It ties event publication to the committed change that gives rise to it, helping ensure that an event is not lost before it enters the flow [23]. The framework developed here becomes directly applicable from that point onward, once questions of preservation, replay, delay, duplication, and settlement arise.

5.3.3 Illustrative design examples

The following examples are not case studies. They are compact design readings of familiar systems, meant to show how the guide can be applied one event-driven process at a time.

5.3.3.1 Food delivery service

A food delivery service usually keeps the order record in an authoritative store, while events and asynchronous handling support courier visibility, restaurant acceptance, kitchen preparation, dispatch, payment, and customer communication. These flows do not all protect the same condition. Courier visibility is chiefly state-oriented. Missed intermediate location updates may be acceptable if newer state quickly restores an adequate view of the courier or order.

Restaurant acceptance, kitchen preparation, and courier dispatch are more time-sensitive. If acceptance arrives too late, the restaurant, courier, customer, or payment context may no longer make the order valid. If preparation or dispatch starts too late, the delivery promise may already have been lost. These processes therefore have a deadline-constrained character: eventual completion is not enough if the action no longer counts when it occurs.

Payment capture, refund issuance, and courier or restaurant payout illustrate a different kind of obligation. Once the relevant business fact exists, the downstream action has to happen reliably and without harmful duplication. A missed refund, a duplicated capture, or a repeated payout is not mainly a stale-state problem or a late-arrival problem. It is a required-effect problem.

In the language of the framework, courier visibility is state-oriented, restaurant acceptance and dispatch-related work are deadline-constrained, and payment, refund, or payout handling are required effects. The handling model should therefore be chosen separately for each process, rather than for the food delivery system as a whole.

5.3.3.2 E-commerce checkout and order processing

An e-commerce system similarly combines several event-driven processes around an authoritative order, payment, and stock record. Customer-facing stock views and many order-status updates are largely state-oriented. They should remain current enough to be useful, but missed intermediate updates may be overtaken by later state while the authoritative record still determines the underlying commitment.

Checkout and payment authorisation are more time-sensitive. Delay or loss at that point can leave the checkout incomplete, invalid, or no longer acceptable to the customer or payment provider. Shipping-related work may also be deadline-constrained where a carrier cutoff, delivery promise, or cancellation window determines whether the action still counts.

Other downstream actions are better read as required effects. Once an order has been created, payment capture, refund issuance, invoice generation, or a fulfilment instruction may have to happen reliably and without harmful duplication. The central concern in those processes is that the action is not lost, repeated, or left to ambiguous later settlement.

In the language of the framework, stock and ordinary order-status views are state-oriented, checkout, payment authorisation, and cutoff-sensitive fulfilment are deadline-constrained, and payment capture, refunds, invoicing, or fulfilment instructions are required effects. The same application therefore contains several reliability questions, each of which calls for its own process-level handling judgment.

Read together, the examples show why the guide is applied process by process rather than to the whole application at once. The relevant question is not whether a system as a whole needs stronger or weaker event handling. It is what each event-driven process must protect, what kind of disturbance threatens that condition, and which handling model is sufficient for that process.

Further short-form examples are summarised in Appendix B. They are included to show the same process-level reading across other familiar domains, without treating those examples as empirical case studies.

5.3.4 Scope and realism

The argument does not depend on treating an application as an event-only system. In many ordinary systems, the authoritative business record remains in a transactional database, while events and asynchronous handling carry meaningful parts of the surrounding work. That is compatible with the scope of this thesis. The question is not whether the database or the event stream is the centre of the system, but which event-driven processes depend on asynchronous handling for correctness-relevant work.

This also defines the guide's scope. It helps examine a particular event-driven process by asking what condition that process must protect, what disturbance threatens it, and which handling model is sufficient. Applied repeatedly, the same discipline can give a clearer view of the system's event processing as a whole, while the judgment remains process-level.

This is a modest form of design guidance. It is most useful where ordinary design language already points to different obligations: something must happen in time,

something must not be lost, something can be retried, or some state can be updated later. The framework gives those distinctions a more explicit shape without claiming to replace the wider architectural judgment needed in a real system.

5.4 Relation to established work

This thesis should be read as a complementary clarification within established work on end-to-end reasoning, coordination, stream processing, idempotence, and recovery. Its contribution is a process-level way of asking what condition a mechanism is meant to protect before judging whether that mechanism is adequate in a given process. This keeps the focus on interpretation and design choice rather than on proposing a new runtime or replacement pattern.

The closest conceptual connection is to end-to-end reasoning. Saltzer, Reed, and Clark argue that a mechanism cannot be judged only from its local position in the system, because correctness often depends on the requirement being protected and on where that requirement can actually be enforced [5]. This thesis applies that discipline to event-driven processes. It asks whether the relevant condition is timely completion, acceptable state, or one downstream action happening reliably and without harmful duplication, and then judges replay, retention, immediate handling, and deferred settlement in relation to that condition.

A related connection is to work on coordination avoidance and CALM. That work asks when coordination is required by relating coordination to the property being protected, such as an application-level invariant or the monotonicity of a computation [8, 9]. The present thesis makes a similar move at the level of event-driven processes, while remaining less formal and more process-facing. It asks which condition the process must protect before deciding whether replay, retention, immediate handling, deferred settlement, idempotence, or producer-side safeguards are relevant.

Work on stream processing and dataflow systems already shows that correctness, latency, and cost have to be balanced rather than maximised in isolation [3, 13]. The thesis expresses that trade-off at the level of a particular event-driven process. In a deadline-constrained process, the useful question is whether preserved work can still settle before expiry. In a state-oriented process, it is whether missed state remains relevant after later updates arrive. In a required-effect process, it is whether the event entered the flow, whether downstream handling can recover it, and whether delayed settlement is an acceptable cost.

The required-effect results connect most directly to work on idempotence, sagas, compensation, and transactional event publication [7, 14, 23]. Those works address important mechanisms and patterns for avoiding repeated effects, coordinating long-running work, or ensuring that events are emitted with the changes that give rise to them. The thesis clarifies how these mechanisms answer different parts of the required-effect problem. Idempotence and deferred settlement matter when repeated effects are harmful. Producer-side safeguards matter when the event might never enter the flow. Replay matters when a downstream handling gap occurs after emission.

The empirical suite should be read in the same restrained way. It is a controlled setting used to make selected distinctions visible: availability is not the same as

timely settlement, final state adequacy is not the same as the relevance of missed state, and downstream handling loss is not the same as source omission. These distinctions are familiar in parts of the existing literature, but the thesis brings them together into a single process-level guide for judging event-driven handling choices.

5.5 Limitations and claim boundaries

The claims made by the thesis are deliberately bounded. The framework is strongest as a way of relating reliability mechanisms to explicit process-level correctness requirements. The three boundary classes are working cases rather than a complete taxonomy. They bring deadline validity, state relevance, and required downstream effects into view, but other correctness concerns may require further classes or orthogonal distinctions.

The empirical work has the same bounded character. The research instrument is a controlled comparison suite rather than a production benchmark. It abstracts from platform heterogeneity, operational scale, and full deployment complexity so that expiry, replay value, repayment, source omission, duplicate pressure, and settlement cost can be compared under fixed conditions. The results should therefore be read as pattern-illuminating evidence within the examined scenarios, not as absolute predictions for production systems.

The design guide has a narrower status. It is a compact articulation of the framework, not a separately validated design method. The present study has not evaluated whether practitioners make better decisions after using the guide, or how it compares with alternative design procedures. Its contribution here is to make the reasoning inspectable: the protected condition is stated, the relevant disturbance is identified, and the handling choice and its trade-off are made explicit.

The thesis also rejects a general ranking of handling models. Retention, replay, immediate handling, and deferred settlement are useful only in relation to the boundary being protected. Deferred settlement, in particular, is not a free improvement. It is appropriate only where the cost of postponing settlement is preferable to the cost of immediate duplicate or repeated downstream effects. The narrower claim defended here is that explicit correctness reasoning gives a clearer basis for judging which handling model is appropriate to the condition a process must protect.

Finally, the framework remains process-level guidance. It can inform broader architecture when applied across the event-driven processes in a system, but architectural judgment still has to account for databases, service boundaries, operational constraints, ownership, monitoring, and recovery practice. The framework disciplines one part of that judgment: the relation between what a process must protect and how its event-driven handling should be arranged.

Chapter 6

Conclusions and Future Work

This thesis has argued that reliability in event-driven processes becomes clearer when the condition a process must protect is stated first. Event-driven systems provide useful ways to preserve, retry, replay, defer, and deduplicate work, but their value depends on the condition the process is meant to protect. A replay path, retained event, immediate consumer, or deferred settlement step means one thing when timely completion is decisive, another when missed state can be superseded, and another when a downstream effect must be carried through reliably and without harmful duplication.

The thesis develops this point as a boundary-first framework. Correctness boundaries state what must hold for a process to remain acceptable, while process-handling models describe how event-driven work is carried, preserved, replayed, or settled. The controlled empirical suite examines that relation in a deliberately restrained setting. Its results support the central claim that the value of a handling model depends on the condition being protected. In deadline-constrained work, replay and retention help only while unsettled work can still be repaid before expiry. In state-oriented work, the value of replay depends on whether missed state remains relevant or has been overtaken by later updates. In required-effect work, downstream replay can recover handling gaps after emission, but not source omission, while deferred settlement reduces immediate duplicate pressure by moving part of the burden into later reconciliation.

The contribution lies in the discipline of relating correctness boundaries to process-handling choices. The thesis connects a correctness condition, a disturbance, a handling model, and a resulting trade-off, and asks how they fit together in a given process. The guide gives this reasoning a more concrete form. Its design value is modest but useful: it helps make the basis for a handling choice clearer by asking first which condition defines correctness for the process, and only then which handling model is sufficient to protect it.

6.1 Future Work

Future work could extend the thesis by moving from the controlled setting toward real event-driven design. The central question is how developers and architects decide what a process must protect before they choose event-handling mechanisms, infrastructure support, and recovery behaviour. The framework can support that inquiry, but the more important object of study is the reasoning itself.

One direction is to examine design practice more closely. Future work could study how teams describe correctness requirements for event-driven processes, whether those requirements are stated explicitly, and how they influence choices about replay, retention, idempotence, settlement, producer safeguards, and monitoring. Such work could also examine incidents involving delay, loss, duplication, omission, or stale state, asking whether the protected condition was clear before the failure occurred.

A second direction is to study the relation between application design and event-streaming infrastructure. Modern brokers, stream processors, and managed messaging services can provide powerful support for retention, replay, ordering, delivery guarantees, schema management, and recovery. They do not, however, remove the need to decide what the application is trying to protect. Future work could examine how much correctness responsibility is carried by the surrounding infrastructure, how much remains in application code, and how teams coordinate the two in practice.

A third direction is to examine richer technical settings. The empirical suite used here is deliberately small, so that the main distinctions remain visible. Later work could study different brokers, stream processors, storage choices, workload shapes, deployment models, and recurring failure modes. The point would not be to turn the framework into a general benchmark. It would be to ask whether the same condition-specific patterns still appear when recovery is shaped by more realistic system conditions.

A fourth direction is to develop stronger operational support for correctness-oriented evidence. Some protected conditions could be connected to runtime checks: whether deadline-sensitive work is still repayable, whether state has become stale or been superseded, whether a required effect is missing or duplicated, or whether settlement has been delayed beyond an acceptable point. This would shift the framework from a design-time clarification toward observable signals that can support operation, review, and incident analysis.

Further work could also examine cases near the edges of the three working boundary classes. Deadline-constrained, state-oriented, and required-effect work bring important obligations into view, but real systems may involve conditions that are harder to place cleanly. Ordering, auditability, relationships across several cases, or groups of effects that must be settled together may complicate the question of what the process must protect. In such cases, the protected condition may need to be stated at a wider scope, or in terms of several connected obligations rather than one primary condition. Future work could study how boundary-first reasoning should be extended when correctness depends on more than one event, one case, or one downstream effect.

Taken together, these directions would move the work from conceptual clarification and controlled demonstration toward a broader study of correctness reasoning in event-driven systems. The lasting question is not whether event-driven systems need stronger handling in general, or whether a particular framework should be adopted. It is how a given process comes to be understood as correct, what can threaten that condition, and how application code and event-streaming infrastructure together preserve it.

33

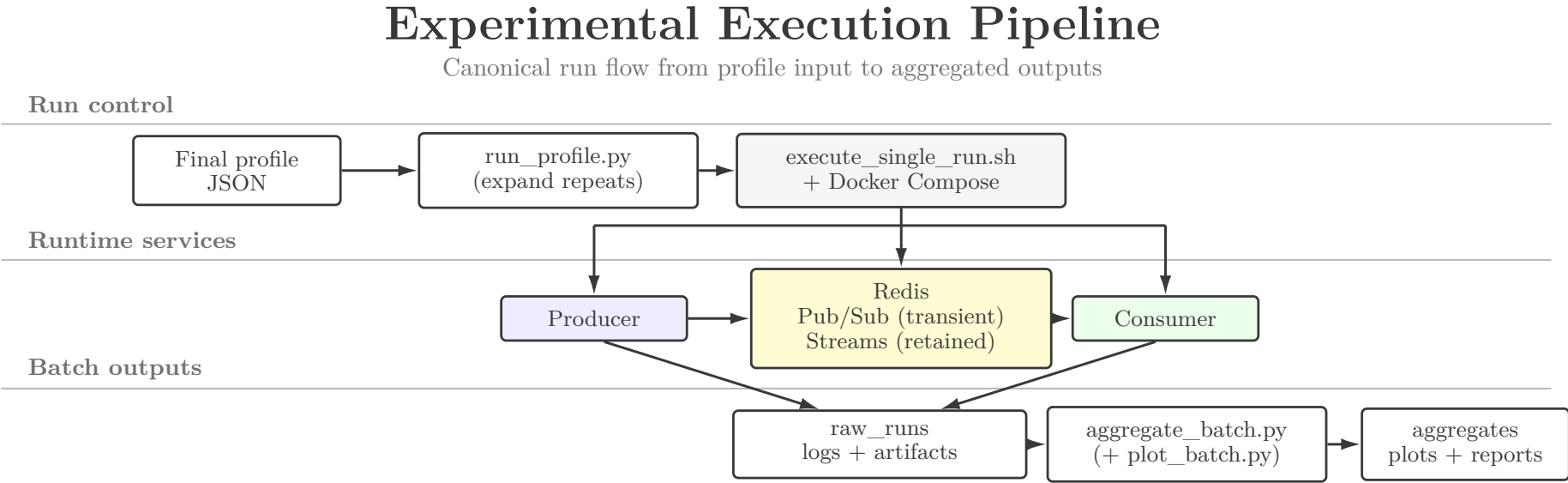


Figure A.1: Experimental execution pipeline from profile input to aggregated outputs.

Appendix B

Supplementary summary table

Table B.1: Design scenario summary

System	State-oriented	Deadline-constrained	Required-effect
Food delivery	Courier visibility	Acceptance and dispatch timing	Payment, refund, or payout handling
Online reservation	Availability view	Confirmation before hold expiry	Reminder and calendar update
E-commerce checkout	Stock and order-status view	Checkout and payment authorisation	Payment capture, refunds, invoicing, or fulfilment instructions
Subscription billing	Account and access-status view	Renewal and status-transition timing	Receipts, reminders, and billing follow-ons
Perishable refrigerated-goods logistics	Temperature and shipment-condition visibility	Intervention before spoilage threshold	Reroute, notification, and compliance action

Each row maps one illustrative example to the three correctness categories used in the thesis.

Table C.1: Boundary-first guide for process-handling choice.

Design check	Process-level question	What the answer indicates	Handling implication
Scope the process	Which specific event-driven process is being judged?	The guide applies to one process at a time, not to the whole application at once.	If several protected conditions appear, split the larger system into separate process-level judgments before choosing a handling model.
State the boundary	What must hold for a case in this process to count as acceptable?	The process is primarily deadline-constrained, state-oriented, or required-effect.	Do not choose replay, retention, immediate handling, or deferred settlement before this condition is clear.
Identify the disturbance	What kind of failure would make the boundary false?	The relevant threat may be delay, backlog, missed handling, stale state, duplicate pressure, delayed settlement, or source omission.	Mechanisms should be judged against the failure they can actually address.
Check deadline sensitivity	Would completion still count if it arrived late?	Timeliness is part of correctness, not merely a performance concern.	Retention and replay help only if preserved work can still settle before expiry. Use immediate or deadline-aware handling when lateness invalidates the case.
Check state relevance	Can later state overtake a missed or delayed update?	Replay matters when the missed update remains relevant to the visible or resulting state.	Transient/immediate may be sufficient for rapidly superseded state. Retained/immediate is stronger where missed state can remain stale or harmful.
Check downstream recovery	Was the event emitted, but missed or delayed downstream?	The failure is inside the consumer-side handling path.	Retained/immediate can recover emitted work through replay when the downstream effect still has to happen.
Check duplicate harm	Would repeated execution of the downstream action be harmful?	Correctness may require not only that the effect happens, but that it does not happen repeatedly in a harmful way.	Retained/deferred may be appropriate when duplicate effects are costly and later settlement is acceptable.
Check settlement tolerance	Is delayed settlement acceptable for this process?	Deferred handling can reduce duplicate execution, but may add reconciliation work and settlement delay.	Use retained/deferred only when the cost of later settlement is preferable to the cost of harmful duplicate effects.
Check source omission	Could the event fail to enter the flow at all?	The disturbance lies before downstream handling. Replay and retention cannot recover what was never emitted.	Treat this as a producer-integrity concern before applying the downstream process-handling guide.
Choose the handling model	Which lowest-sufficient handling model protects the stated boundary under the likely disturbance?	The model should fit the boundary, not appear strong in the abstract.	Choose transient/immediate, retained/immediate, or retained/deferred according to what the process must protect. Record the residual risk and the metric that would reveal failure.

Appendix D

Supplementary method tables

Table D.1: Operational definitions of reported metrics.

Metric	Computation rule	Used for	Interpretation
Deadline miss rate	Expired cases divided by produced cases.	Deadline-constrained	Share of cases that expired before deadline-valid completion.
Completion-in-time rate	Cases completed before expiry divided by produced cases.	Deadline-constrained	Share of cases that reached a valid outcome in time.
Unresolved burden	Produced cases not yet attained at a runtime sample.	Deadline-constrained	Work still unsettled during the run.
Doomed burden	Unsettled produced cases whose deadline has already passed at a runtime sample.	Deadline-constrained	Work that remains unresolved after timely correctness is no longer possible.
Repayment pattern	Reduction of unresolved burden before cases cross the deadline, read with completion-in-time and doomed burden.	Deadline-constrained	Shows whether accumulated work can still be worked down in time.
Resolution pulse	Completions and expiries counted per fixed runtime bin.	Deadline-constrained	Burstiness of settlement activity during recovery.
Required-effect attainment	Cases whose required downstream effect occurred divided by produced cases.	Required-effect	Share of required effects that happened.
Received-event count	Events observed by the consumer after omission or duplicate pressure.	Required-effect	Separates downstream loss from source omission and duplicate triggering.
Duplicate side-effect count	Repeated downstream executions beyond the intended effect.	Required-effect	Measures harmful duplicate execution.
Correction rewrites and reconciliation passes	Deferred overwrites and settlement sweeps recorded by the retained/deferred model.	Required-effect	Measures the cost introduced by duplicate containment.
Settlement delay	Time from producer completion to last recorded attainment.	Required-effect	Shows whether deferred handling shifts cost into later settlement.
Latest-state attainment rate	Cases whose resulting latest state is acceptable divided by produced cases.	State-oriented	Measures final state adequacy.
Outage-exposed seen fraction	Seen outage-relevant updates divided by expected outage-relevant updates.	State-oriented	Separates final adequacy from outage-time exposure.

Table D.2: Scenario and parameter specification.

Module	Scenario family	Cells	Workload	Boundary parameter	Disturbance specification	Handling models	
Deadline-constrained	Baseline	3	1000 cases, 5 ms emission interval.	1.2 s deadline window, 30 s observation window.	No disturbance.	T/I, R/D.	R/I,
Deadline-constrained	Backlog shock	3	1000 cases, 5 ms base emission interval.	1.2 s deadline window, 35 s observation window.	2.8 s interruption triggered at 20% progress, with overload emission interval of 1 ms for 6 s starting at 1.5 s.	T/I, R/D.	R/I,
Deadline-constrained	Moderate degradation	3	1000 cases, 5 ms emission interval.	1.2 s deadline window, 35 s observation window.	Sustained 8 ms processing delay over 8.0 s.	T/I, R/D.	R/I,
Deadline-constrained	High degradation	3	1000 cases, 5 ms emission interval.	1.2 s deadline window, 35 s observation window.	Sustained 50 ms processing delay over 8.0 s.	T/I, R/D.	R/I,
Required-effect	Baseline	3	1000 logical cases, 5 ms emission interval.	Required effect, no deadline window.	No disturbance.	T/I, R/D.	R/I,
Required-effect	Handling gap, standard	3	1000 logical cases, 5 ms emission interval.	Required effect, no deadline window.	3.5 s handling interruption starting at 2.0 s.	T/I, R/D.	R/I,
Required-effect	Handling gap, extreme	3	1000 logical cases, 5 ms emission interval.	Required effect, no deadline window.	5.0 s handling interruption starting at 1.5 s.	T/I, R/D.	R/I,
Required-effect	Source omission	3	1000 logical cases, 5 ms emission interval.	Required effect, no deadline window.	20% of logical cases omitted before emission, leaving 800 emitted cases.	T/I, R/D.	R/I,
Required-effect	Duplicate pressure, standard	3	1000 logical cases, 5 ms emission interval.	Required effect, no deadline window.	40% duplicate-selected cases, two duplicate repeats, 1 ms duplicate delay, 2 ms jitter.	T/I, R/D.	R/I,
Required-effect	Duplicate pressure, extreme	3	1000 logical cases, 5 ms emission interval.	Required effect, no deadline window.	70% duplicate-selected cases, two duplicate repeats, 1 ms duplicate delay, 2 ms jitter.	T/I, R/D.	R/I,
State-oriented	Backlog shock	2	300 logical cases, 3 updates per entity, 0.3 obsolete replay fraction.	Latest-state target version 3.	2.4 s interruption triggered at 80% progress.	T/I, R/I.	
State-oriented	Backlog shock with forward resume	2	300 logical cases, 4 updates per entity, 0.3 obsolete replay fraction.	Latest-state target version 4.	0.8 s interruption triggered at 60% progress, followed by forward resumption.	T/I, R/I.	

In retained/deferred runs, the reconciliation interval was 250 ms in the required-effect module, 750 ms for deadline backlog shock, and 1600 ms for deadline degradation scenarios. The state-oriented module compares transient/immediate and retained/immediate only, because the question examined there is replay relevance for state adequacy and outage-time exposure rather than deferred settlement.

Table D.3: Disturbance-to-channel map.

Scenario family	Primary disturbance locus	Observable effect	Replay expectation
Handling gap	Downstream handling interruption.	Events are emitted, but missed or delayed by the consumer.	High for retained modes when the event remains available.
Source omission	Producer-side non-emission.	Some logical cases never enter the event flow.	None. Replay cannot recover an event that was never emitted.
Backlog shock	Temporary processing interruption and catch-up pressure.	Work accumulates and then resolves in bursts.	Depends on whether the remaining validity or relevance window is still open.
Degradation	Sustained processing slowdown.	Work progresses under continuous latency pressure.	Limited by deadline windows and accumulated doomed burden.
Duplicate pressure	Repeated triggering of the same obligation.	Duplicate downstream execution risk increases.	Deferred handling can absorb repeated triggers before settlement.
State backlog shock	Outage over a state-relevant update window.	State may be unseen, lost, or restored by later updates.	High when the missed state remains relevant to correctness.

Table D.4: Evaluation controls and scope protection.

Control objective	Operational check	Evidence in the suite	Residual limitation
Pattern stability	Each scenario cell was repeated ten times.	34 cells and 340 completed runs.	Single-suite setting, not an external validity claim.
Channel separation	Source omission and handling gaps are separate scenario families.	Required-effect module.	Does not model every producer or broker failure mode.
Boundary discipline	Deadline, state, and required-effect modules are reported separately.	Module structure and metric groups.	Larger applications may contain several process-level obligations.
Traceability	Raw runs, run specifications, metric summaries, aggregate outputs, and plots are preserved.	Batch folder structure.	Reproduction still depends on the local Docker/Redis runtime.
Interpretive restraint	Results are read as controlled comparisons.	Method, Results, and Limitations wording.	No claim of platform benchmarking or universal quantitative law.

References

- [1] Laigner, R., Almeida, A. C., Assuncao, W. K. G., and Zhou, Y. An empirical study on challenges of event management in microservice architectures. *ACM Transactions on Software Engineering and Methodology*, advance online publication, 2025. DOI: 10.1145/3776581.
- [2] Laigner, R., Zhou, Y., Salles, M. A. V., Liu, Y., and Kalinowski, M. Data management in microservices: State of the practice, challenges, and research directions. *Proceedings of the VLDB Endowment*, 14(13), 3348–3361, 2021. DOI: 10.14778/3484224.3484232.
- [3] Akidau, T., Bradshaw, R., Chambers, C., Chernyak, S., Fernandez-Moctezuma, R. J., Lax, R., McVeety, S., Mills, D., Perry, F., Schmidt, E., and Whittle, S. The Dataflow model: A practical approach to balancing correctness, latency, and cost in massive-scale, unbounded, out-of-order data processing. *Proceedings of the VLDB Endowment*, 8(12), 1792–1803, 2015. DOI: 10.14778/2824032.2824076.
- [4] Alpern, B., and Schneider, F. B. Defining liveness. *Information Processing Letters*, 21(4), 181–185, 1985. DOI: 10.1016/0020-0190(85)90056-0.
- [5] Saltzer, J. H., Reed, D. P., and Clark, D. D. End-to-end arguments in system design. *ACM Transactions on Computer Systems*, 2(4), 277–288, 1984. DOI: 10.1145/357401.357402.
- [6] Saito, Y., and Shapiro, M. Optimistic replication. *ACM Computing Surveys*, 37(1), 42–81, 2005. DOI: 10.1145/1057977.1057980.
- [7] Garcia-Molina, H., and Salem, K. Sagas. *Proceedings of the ACM SIGMOD International Conference on Management of Data*, 249–259, 1987. DOI: 10.1145/38713.38742.
- [8] Bailis, P., Fekete, A., Franklin, M. J., Ghodsi, A., Hellerstein, J. M., and Stoica, I. Coordination avoidance in database systems. *Proceedings of the VLDB Endowment*, 8(3), 185–196, 2014. DOI: 10.14778/2735508.2735509.
- [9] Hellerstein, J. M., and Alvaro, P. Keeping CALM: When distributed consistency is easy. *Communications of the ACM*, 63(9), 72–81, 2020. DOI: 10.1145/3369736.
- [10] Eugster, P. T., Felber, P. A., Guerraoui, R., and Kermarrec, A.-M. The many faces of publish/subscribe. *ACM Computing Surveys*, 35(2), 114–131, 2003. DOI: 10.1145/857076.857078.
- [11] Patterson, D., Brown, A., Broadwell, P., Candea, G., Chen, M., Cutler, J., Enriquez, P., Fox, A., Kiciman, E., Merzbacher, M., Oppenheimer, D., Sastry, N., Tetzlaff, W., Traupman, J., and Treuhaft, N. Recovery Oriented Computing

- (ROC): Motivation, definition, techniques, and case studies. UCB/CSD-02-1175, University of California, Berkeley, 2002.
- [12] Dean, J., and Barroso, L. A. The tail at scale. *Communications of the ACM*, 56(2), 74–80, 2013. DOI: 10.1145/2408776.2408794.
 - [13] Fragkoulis, M., Carbone, P., Kalavri, V., and Katsifodimos, A. A survey on the evolution of stream processing systems. *The VLDB Journal*, 33(2), 507–541, 2024. DOI: 10.1007/s00778-023-00819-8.
 - [14] Helland, P. Idempotence is not a medical condition. *ACM Queue*, 10(4), 30–46, 2012. DOI: 10.1145/2181796.2187821.
 - [15] Vogel, A., Henning, S., Perez-Wohlfeil, E., Ertl, O., and Rabiser, R. A comprehensive benchmarking analysis of fault recovery in stream processing frameworks. *Proceedings of the 18th ACM International Conference on Distributed and Event-Based Systems*, 171–182, 2024. DOI: 10.1145/3629104.3666040.
 - [16] Tahir, J., Mayer, R., Doblander, C., and Jacobsen, H.-A. How reliable are streams? End-to-end processing-guarantee validation and performance benchmarking of stream processing systems. *Proceedings of the VLDB Endowment*, 18(3), 585–598, 2025. DOI: 10.14778/3712221.3712227.
 - [17] Peffers, K., Tuunanen, T., Rothenberger, M. A., and Chatterjee, S. A design science research methodology for information systems research. *Journal of Management Information Systems*, 24(3), 45–77, 2007. DOI: 10.2753/MIS0742-1222240302.
 - [18] Hevner, A. R., March, S. T., Park, J., and Ram, S. Design science in information systems research. *MIS Quarterly*, 28(1), 75–105, 2004. DOI: 10.2307/25148625.
 - [19] Docker documentation. Docker Compose. Available at: <https://docs.docker.com/compose/>.
 - [20] Redis documentation. Redis Pub/Sub. Available at: <https://redis.io/docs/latest/develop/pubsub/>.
 - [21] Redis documentation. Redis Streams. Available at: <https://redis.io/docs/latest/develop/data-types/streams/>.
 - [22] Shapiro, M., Pregelica, N., Baquero, C., and Zawirski, M. Conflict-free replicated data types. *Proceedings of the 13th International Symposium on Stabilization, Safety, and Security of Distributed Systems*, 386–400, 2011. DOI: 10.1007/978-3-642-24550-3_29.
 - [23] AWS Prescriptive Guidance. Transactional outbox pattern. Amazon Web Services, updated 2024. Available at: <https://docs.aws.amazon.com/prescriptive-guidance/latest/cloud-design-patterns/transactional-outbox.html>.